

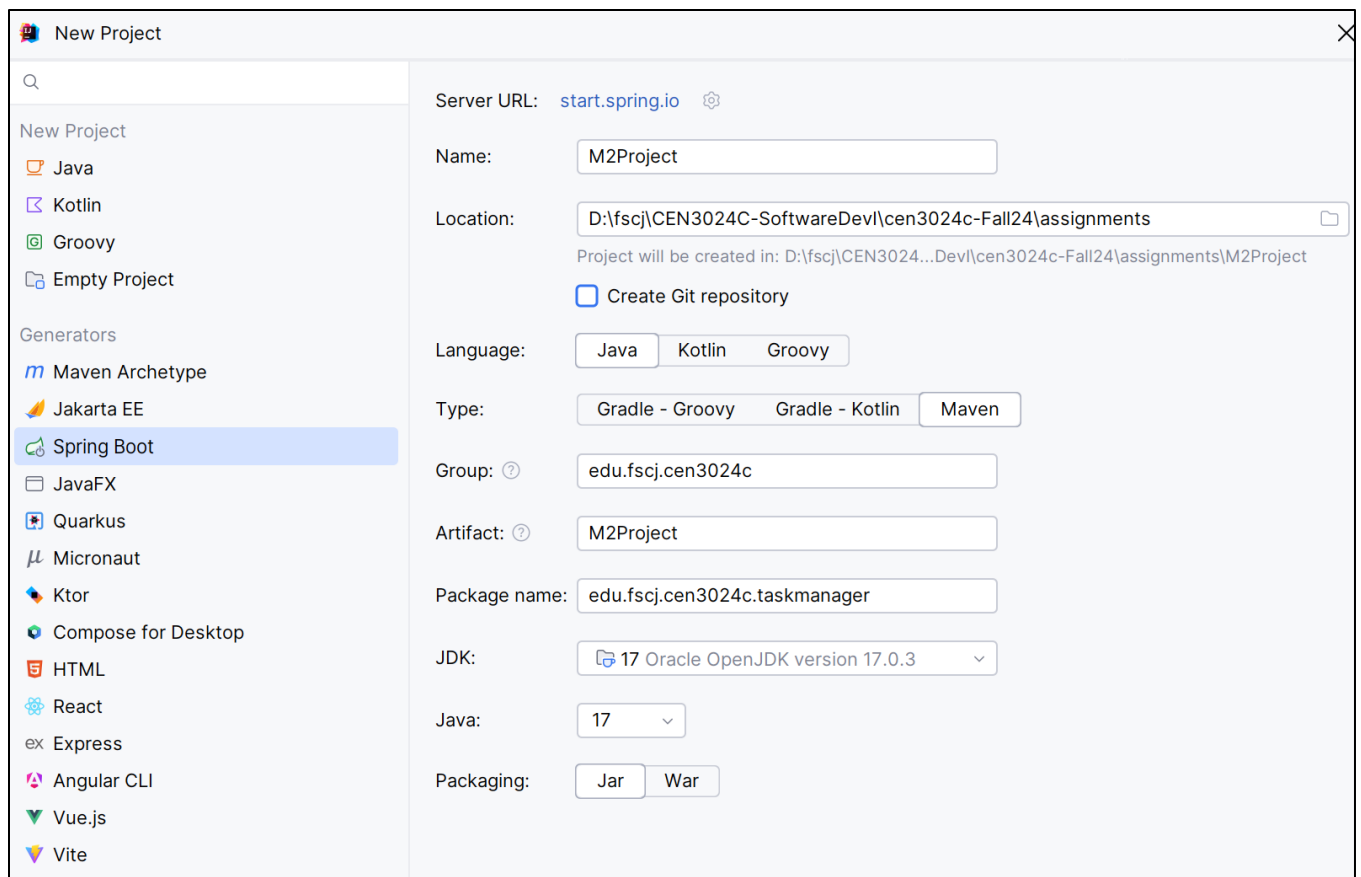
## CEN3024C Module 3 Project

For this project we will practice using PostgreSQL and IntelliJ w/Spring Data JPA to create a simple task management application which includes a database, repository class, service class, and controller class with API endpoints. The application will use Spring Web which provides a basic MVC configuration.

Note: In addition to specifying the Spring Boot dependencies show below, you will need to load the “JPA Buddy” plug-in in IntelliJ as follows: File > Settings > Plugins; search for "JPABuddy" then restart IntelliJ.

- Start by creating your project using the IntelliJ Spring Boot generator. Select Java and Maven. Your group should be the “upper level” of your package (e.g., edu.fscj.cen3024c) and the package should extend that to indicate the project.
- Select Java 17 and Jar packaging.

**Note: the images in this document use “M2Project”; these should be changed to “M3Project” wherever applicable.**



Spring Boot Dependencies:  
Developer Tools/Spring Boot DevTools

(provides development-time features like automatic restarts, live reload, and faster application restarts)

Web/Spring Web

SQL/Spring Data JPA

SQL/PostgreSQL Driver

Spring Boot: 3.3.3

☒ Download pre-built shared indexes for JDK and Maven libraries

Dependencies:

Q Search

Developer Tools

☐ GraalVM Native Support

☐ GraphQL DGS Code Generation

☒ Spring Boot DevTools

☐ Lombok

☐ Spring Configuration Processor

☐ Docker Compose Support

☐ Spring Modulith

Web

☒ Spring Web

☐ Spring Reactive Web

☐ Spring for GraphQL

☐ Rest Repositories

☐ Spring Session

☐ Rest Repositories HAL Explorer

☐ Spring HATEOAS

☐ Spring Web Services

☐ Jersey

☐ Vaadin

☐ Netflix DGS

☐ htmx

Spring Boot DevTools

Provides fast application restarts, LiveReload, and configurations for enhanced development experience.

Added dependencies:

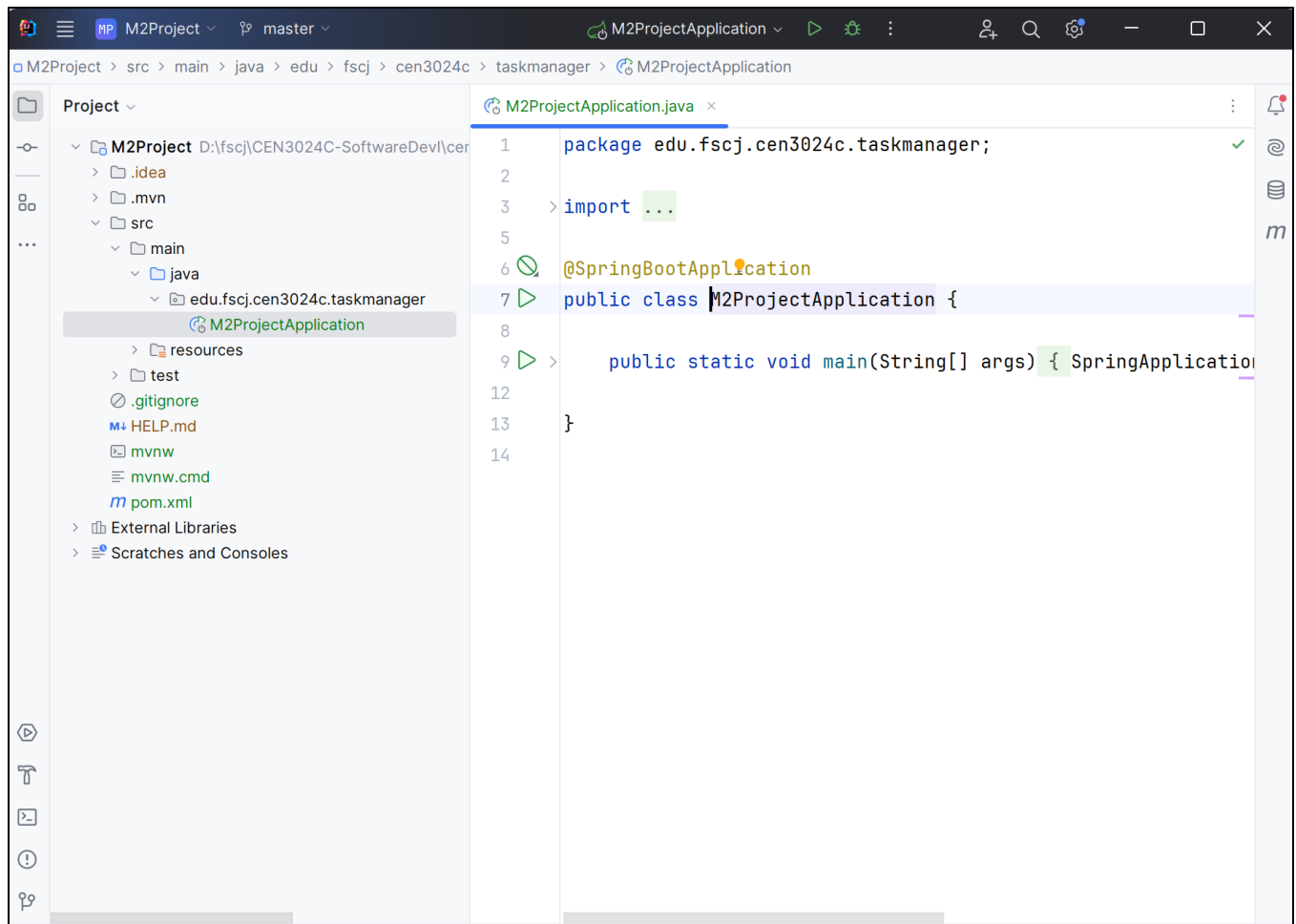
× Spring Data JPA

× Spring Web

× PostgreSQL Driver

× Spring Boot DevTools

IntelliJ creates the application class for you:



### Here are some PostgreSQL tips from a previous semester:

Names which use upper case must be in double quotes

Use lower case (default) for table names so JPA/Hibernate annotation works

c:"Program Files"\PostgreSQL\16\bin\psql -U postgres -d dbname

(postgres is default DB)

or just

c:"Program Files"\PostgreSQL\16\bin\psql -U postgres

(to create/drop a DB)

P@ssw0rd1 (on Horizon)

\l to list databases

\c dbname to connect to a db

\dt to list tables and relationships (reports everything as "relations")

\dt+ for extended details

\dn to list schemas

\q to quit

- Create the database using psql:

```
D:\> c:\"Program Files"\PostgreSQL\16\bin\psql -U postgres -d postgres
Password for user postgres
```

```
psql (16.1)
```

```
WARNING: Console code page (437) differs from Windows code page (1252)
        8-bit characters might not work correctly. See psql reference
        page "Notes for Windows users" for details.
```

```
Type "help" for help.
```

```
postgres=# \l
```

```
    List of databases
```

| Name     |  | Owner    | ... |
|----------|--|----------|-----|
| postgres |  | postgres | ... |
| ...      |  |          |     |

(3 rows)

```
postgres=# CREATE DATABASE taskmanager_db;
CREATE DATABASE
```

```
postgres=# \l
```

```
    List of databases
```

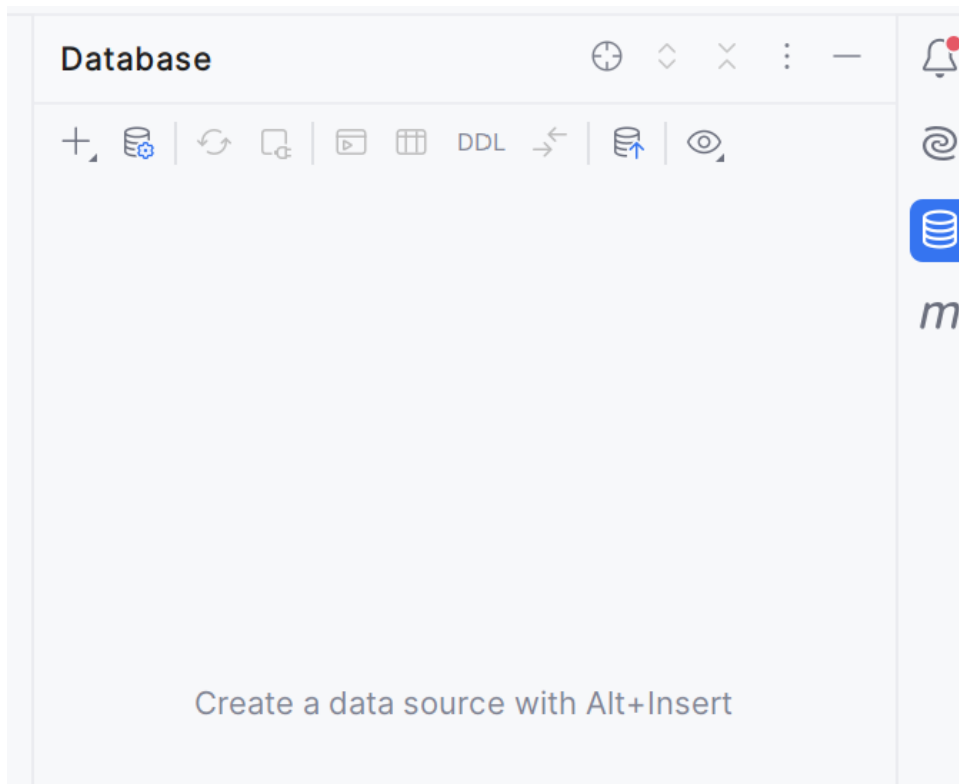
| Name           |  | Owner    | ... |
|----------------|--|----------|-----|
| postgres       |  | postgres | ... |
| taskmanager_db |  | postgres | ... |
| ...            |  |          |     |

(4 rows)

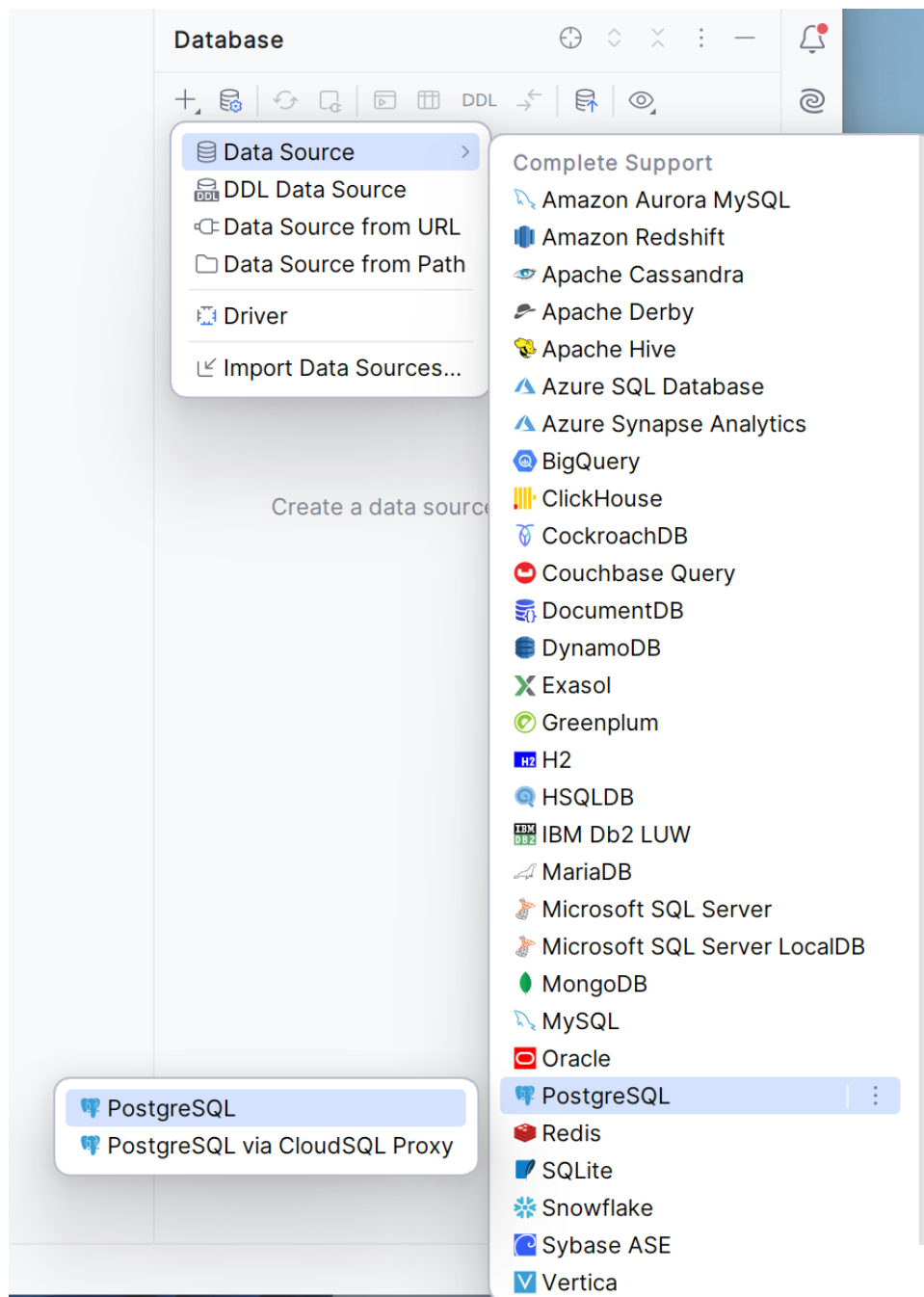
```
postgres=# \q
```

```
D:\fscj\CEN3024C-SoftwareDevI\cen3024c-Fall124\assignments>
```

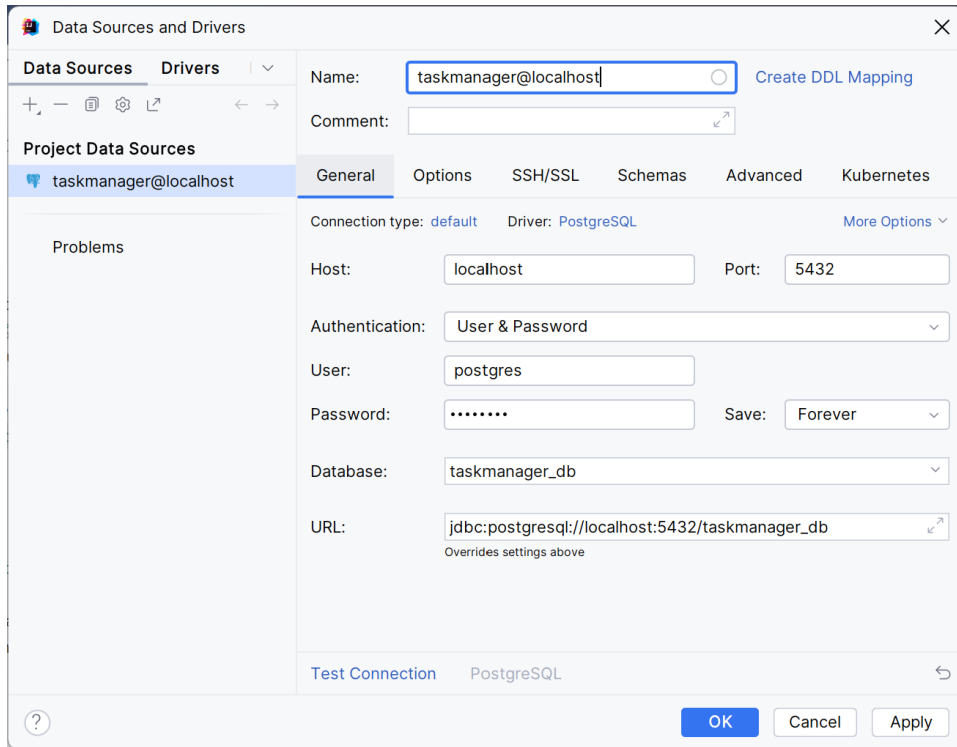
- Open the Database tool in IntelliJ:



- Create a new data source using PostgreSQL:



(**note:** the Name: field refers to the name of the connection; specify the actual database name in the Database field)



- Your application.properties file under the src/main/resources folder should look like this:

spring.application.name=M3Project

spring.jpa.hibernate.ddl-auto=create-drop

spring.jpa.open-in-view=false

spring.datasource.url=jdbc:postgresql://localhost:5432/taskmanager\_db

spring.datasource.username=postgres

spring.datasource.password=postgres # change as necessary

- Create the following table in the new taskmanager\_db database:

**Table Name: task**

**Columns:**

1. **id** (integer, primary key, serial (auto-generated)):

**Create**

task\_id\_pk

Name: task\_id\_pk

Comment:

☒ Primary ☐ Deferrable ☐ Initially Deferred

Columns: id

**Preview**

```
create table task
(
  id serial
  constraint task_id_pk
  primary key
);
```

2. **title** (varchar, not null):

- Description: a string representing the task's title
- Example: "Complete project proposal", "Review design document"

**Modify**

task taskmanager\_db.public [task]

columns 2

id integer = nextval('task\_id\_...')

title VARCHAR

Name: title

Comment:

Data Type: VARCHAR

☒ Not Null

Default Expression:

Identity Kind: ☐ Identity

Min: Max: Step:

**Preview**

```
alter table task
add title VARCHAR not null;
```

3. **description** (varchar, nullable):

- Description: a string representing the description of the task
- Example: "Prepare slides for the meeting" or NULL

The 'Modify' dialog for the 'task' table (taskmanager\_db.public [task]) shows the configuration for the 'description' column. The left sidebar lists the table's structure: columns (3), keys (1), foreign keys, indexes (1), checks, virtual columns, and virtual foreign keys. The 'description' column is selected. The main panel shows the following settings:

- Name: description
- Comment: (empty)
- Data Type: VARCHAR
- Not Null: ☐
- Default Expression: (empty)
- Identity Kind: (empty)
- Identity: ☐
- Min: (empty)
- Max: (empty)
- Step: (empty)

The 'Preview' section at the bottom shows the SQL command:

```
alter table task
add description VARCHAR;
```

4. **status** (varchar w/Check, not null):

- Description: the current status of the task, with three possible values: PENDING, IN\_PROGRESS, or COMPLETED.

The 'Modify' dialog for the 'task' table (taskmanager\_db.public [task]) shows the configuration for the 'status' column and the 'task\_status\_check' constraint. The left sidebar lists the table's structure: columns (4), keys (1), foreign keys, indexes (1), checks (1), virtual columns, and virtual foreign keys. The 'status' column is selected. The main panel shows the following settings:

- Name: task\_status\_check
- Comment: (empty)
- Deferable: ☐
- Initially Deferred: ☐
- No Inherit: ☐
- Predicate: (empty)

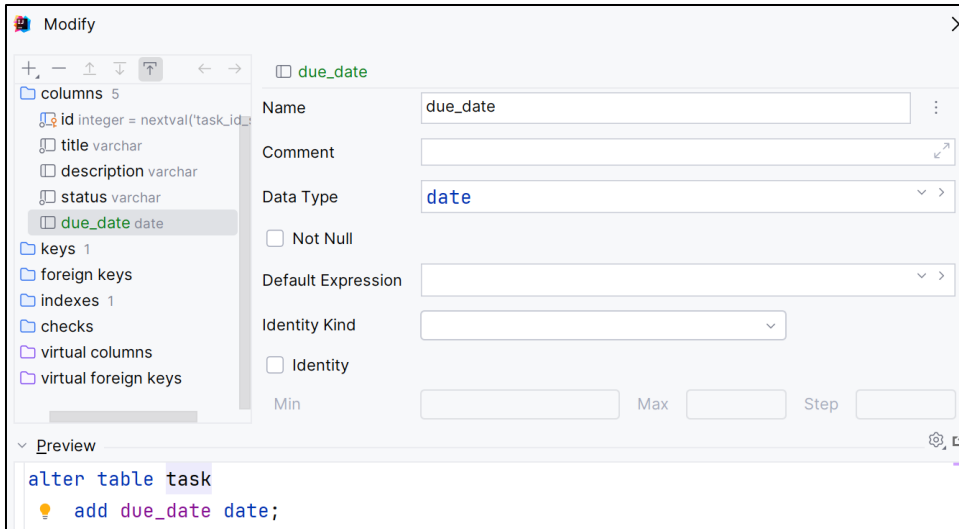
The 'Preview' section at the bottom shows the SQL commands:

```
alter table task
add status VARCHAR not null;

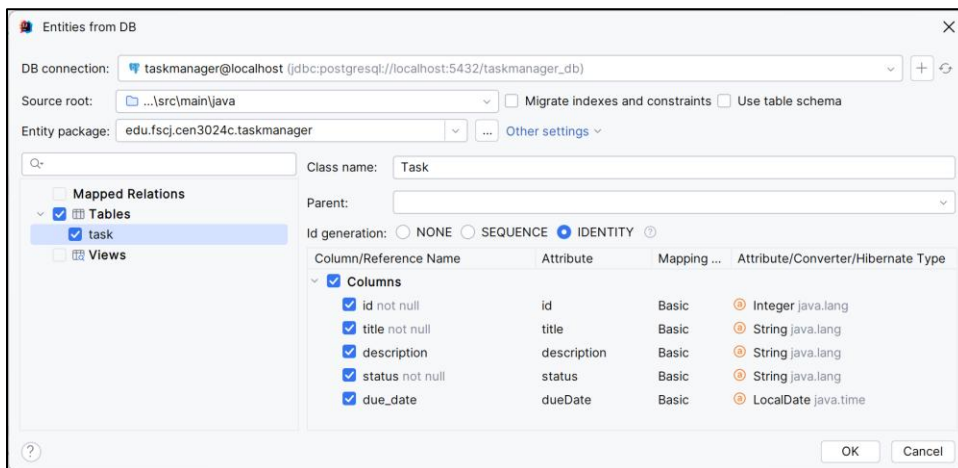
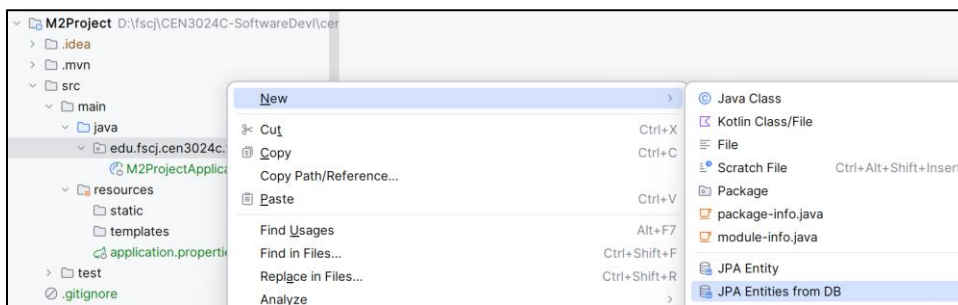
alter table task
add constraint task_status_check
check (status IN ('PENDING', 'IN_PROGRESS', 'COMPLETED'));
```

5. **due\_date** (date, Nullable):

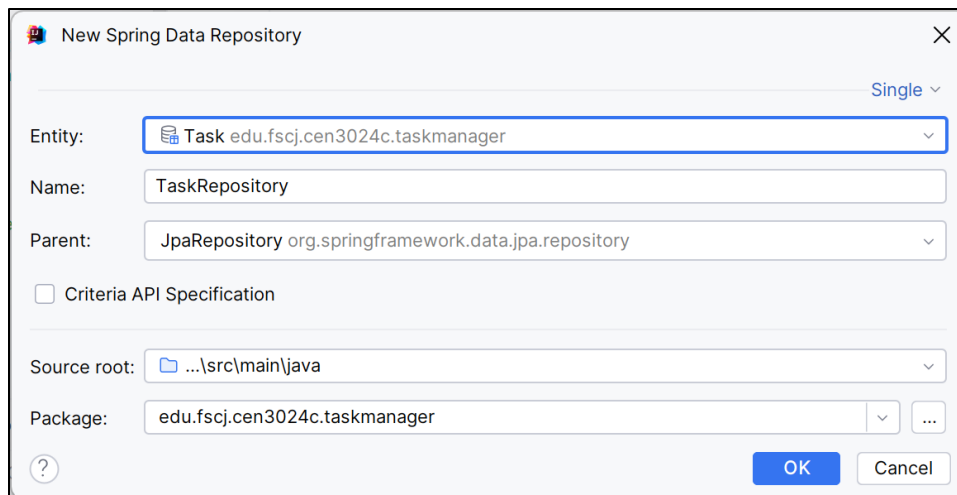
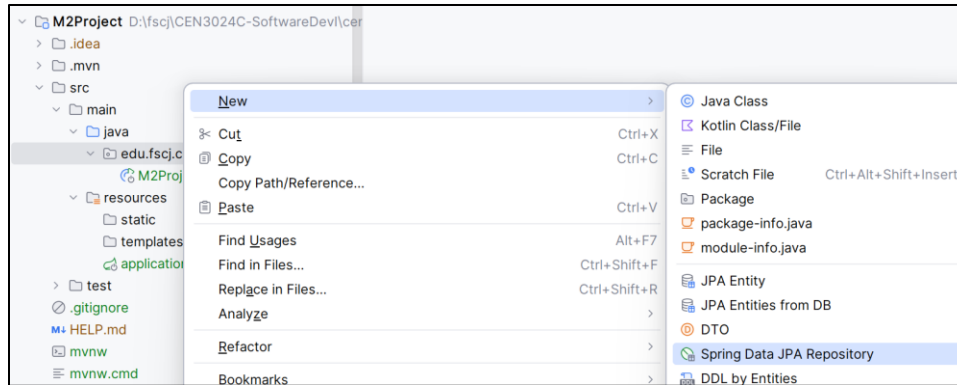
- PostgreSQL Type: date
- Description: the due date for the task in the format YYYY-MM-DD



- Create a JPA Entity from the DB (this class will represent the database table in the application):



- Implement a TaskRepository interface that extends JpaRepository (an interface in Spring Data that provides CRUD operations for JPA entities without boilerplate code):



- Modify the TaskService class so it uses the DB instead of the local hashtable.

```
package edu.fscj.cen3024c.taskmanager;
```

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
```

```
import java.util.List;
```

```
@Service
```

```
public class TaskService {
```

```
    @Autowired
```

```
    private TaskRepository taskRepository;
```

```
    public List<Task> findAll() {
        return taskRepository.findAll();
    }
```

```
    public Task findById(Integer id) {
```

```

        return taskRepository.findById(id)
            .orElseThrow(() -> new TaskNotFoundException(id));
    }

    public Task save(Task task) {
        return taskRepository.save(task);
    }

    public void deleteById(Integer id) {
        taskRepository.deleteById(id);
    }
}

```

- The TaskController from Module 2 should be usable without modification.
- Start the application and use the Module 2 cURL commands (provided in the Module 3 project as a .bat file) in a Command Tool window to test the API endpoints. Take screen snips of results of executing the following and upload to your assignment repo.
- Submit your project to your assignment repo if using GitHub, otherwise zip your project and submit to Canvas; be sure include the screen snips with the CURL command results.